

Sprzętowa implementacja funkcji aktywacji ReLU, sigmoid i tanh

Jakub Jucha Patryk Kowalczyk

Czerwiec 2026

Spis treści

1	Wprowadzenie	4
2	Skrótowa lista potencjalnych metod	8
2.1	ReLU	8
2.2	Sigmoid i tanh	8
3	Szczegółowy opis metod	9
3.1	LUT	9
3.2	LUT + interpolacja liniowa / wielomianowa	9
3.3	Algorytm CORDIC (Coordinate Rotation Digital Computer)	10
3.4	Szereg Taylora	12
3.5	Aproksymacja wielomianami Czebyszewa	13
3.6	Aproksymacja odcinkowo-liniowa (PWL - Piecewise Linear Approximation)	14
3.7	Aproksymacja odcinkowo-wielomianowa (PWP - Piecewise Polynomial Approximation)	15
3.8	Metoda PLAN (Piecewise Linear Approximation of Nonlinear Functions)	16
4	Implementacja	18
4.1	ReLU	18
4.2	tanh	18
5	Github	20
6	Testbench	21
6.1	ReLU	21
6.2	tanh	21
7	Porównywanie metod	22
7.1	Planowane środowisko symulacyjne i model referencyjny	22
7.2	Kryteria oceny	22
7.2.1	Dokładność (Analiza błędów)	23
7.2.2	Zużycie zasobów FPGA	23

7.2.3	Szybkość działania	23
7.3	Narzędzia w EDA Playground	24
7.3.1	Symulatory (ang. simulators)	24
7.3.2	Synteza (ang. synthesizers)	24
7.3.3	VTR 7.0	25
7.4	Co sprawdzane w czym	25
7.4.1	W Pythonie z Cocotb	25
7.4.2	W Siemens Precision	25
7.4.3	W VTR 7.0 (tylko dla prostego ReLU)	26
8	Wyniki	27
8.1	ReLU	27
8.2	tanh	28
9	Wnioski	29

1 Wprowadzenie

Główną zaletą procesorów (CPU) jest ich uniwersalność — są one w stanie obsłużyć dowolną funkcję matematyczną. Jednak w systemach wbudowanych i akceleratorach sprzętowych stosuje się inne techniki. Funkcja aktywacji utrwalona bezpośrednio w strukturze krzemowej może zwracać wynik już po jednym cyklu zegara, co czyni takie podejście znacznie korzystniejszym pod względem efektywności energetycznej. Istotną wadą tego rozwiązania jest konieczność wydzielenia dedykowanej powierzchni na krzemie oraz brak możliwości późniejszej modyfikacji za pomocą zwykłej aktualizacji oprogramowania [5]. Warto jednak zauważyć, że w przypadku układów FPGA ten problem elastyczności nie występuje.

Najprostszą do zaimplementowania w języku Verilog jest funkcja ReLU (ang. *Rectified Linear Unit*), opisana wzorem $y = \max(0, x)$. Jej realizacja sprzętowa wymaga jedynie komparatora znaku oraz prostego multipleksera: gdy sygnał wejściowy jest ujemny, układ zwraca 0, natomiast w przeciwnym wypadku sygnał wejściowy jest przepuszczany bezpośrednio na wyjście [5].

Kolejną popularną metodą jest przybliżanie wartości funkcji za pomocą tablic przeglądowych LUT (ang. *Look Up Table*), czyli poprzez zapisanie predefiniowanych wartości wejść i wyjść w pamięci. Zaletą tego podejścia jest wysoka szybkość działania oraz możliwość szybkiej podmiany zawartości tablicy na inną. Wadą jest natomiast fakt, że przy wymaganej wyższej precyzji obliczeń rozmiar tablicy drastycznie rośnie [5].

Pewnym ulepszeniem tej koncepcji są tablice LUT z interpolacją. Metoda ta również opiera się na strukturze tablicowej, ale wykorzystuje dodatkowe operacje arytmetyczne do estymacji wyników pomiędzy węzłami tablicy, co pozwala poprawić dokładność bez nadmiernego zwiększania zużycia pamięci. Najprostszą formą interpolacji jest interpolacja liniowa ale dokładniejsze wyniki można uzyskać stosując na przykład interpolację wielomianową. Istnieją również architektury wykorzystujące dwie tablice: jedną o grubszej, a drugą o drobniejszej ziarnistości, których wyniki są sumowane w celu uzyskania ostatecznego przybliżenia [1].

W aplikacjach cyfrowego przetwarzania sygnałów (DSP), które wymagają wyzna-

czania funkcji nieliniowych, takich jak tangens hiperboliczny ($\tanh$), szeroko stosowany jest algorytm CORDIC (ang. *Coordinate Rotation Digital Computer*). Jest to metoda iteracyjna, która wykorzystuje wyłącznie prostą logikę przesunięć bitowych oraz dodawania (ang. *shift-add*) do osiągnięcia bardzo dokładnych wyników. Przykładowa implementacja CORDIC dla funkcji sinus i cosinus została przedstawiona pod adresem <https://zipcpu.com/dsp/2017/08/30/cordic.html>. Większość wiodących dostawców układów FPGA oferuje również gotowe bloki (ang. *IP cores*) dla jednostek CORDIC [1].

Innym podejściem są szeregi Taylora (ang. *Taylor series*) oraz przybliżenia wielomianowe (ang. *polynomial approximations*). Metody te wykorzystują tradycyjne rozwinięcia funkcji w szeregi potęgowe i próbują zaimplementować operacje na zmiennych wyższych rzędów (czyli wyższa potęga tej samej zmiennej) w wielomianach, co jednak wiąże się z koniecznością stosowania zasobożernych układów mnożących [1]. Przykładem przybliżenia wielomianowego jest aproksymacja wielomianami Czebyszewa [7].

Aproksymacja odcinkowa (PWL - Piecewise Linear Approximation) jest kolejną metodą, która polega na podziale dziedziny funkcji na segmenty i aproksymacji każdego z nich za pomocą prostej linii [6]. Dla każdego segmentu wyznacza się równanie prostej w formie $p(x) = kx + b$, gdzie k to nachylenie (slope), a b to punkt przecięcia (intercept). W obszarach nasycenia (np. dla $x > 8$ lub $x < -8$) funkcja jest przybliżana wartościami stałymi (odpowiednio 1 lub 0). System składa się z komparatorów, mnożników i sumatorów. Komparatory sprawdzają, w którym przedziale znajduje się wartość wejściowa x , a multiplexer (MUX) wybiera ostateczny wynik z odpowiedniej ścieżki obliczeniowej [4].

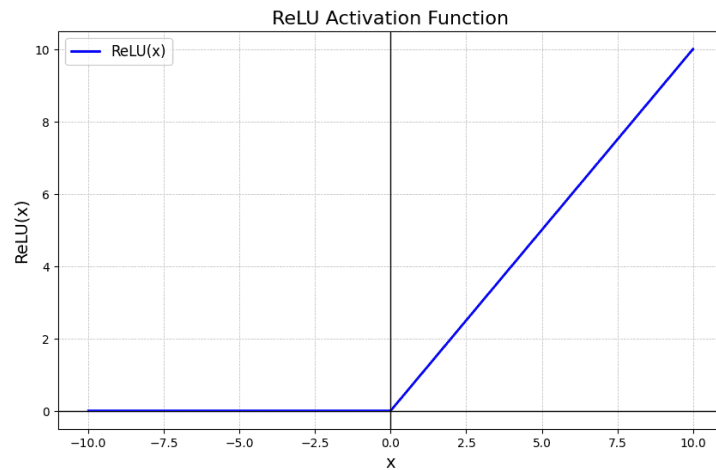
Ulepszeniem tej metody jest PLAN (Piecewise Linear Approximation of Nonlinear Functions). W klasycznej aproksymacji liniowej ($Y = aX + b$) konieczne jest użycie mnożników, które w sprzęcie są kosztowne i wolne. Algorytm PLAN rozwiązuje ten problem poprzez specyficzny dobór nachyleń (gradientów) linii. Nachylenia są dobierane tak, aby operację mnożenia można było zastąpić prostymi przesunięciami bitowymi (ang. *shift operations*). Algorytm dzieli dziedzinę funkcji na konkretne zakresy, wykorzystując symetrię sigmoidy względem punktu $X=0$ (gdzie $Y=0,5$). Dzięki temu obliczenia wykonuje się tylko dla wartości bezwzględnej $|X|$, a dla wartości ujemnych wynik jest odpowiednio przekształcany ($1-Y$) [3].

PPA (PWP) (Piecewise Polynomial Approximation) to rozwinięcie idei PWL. Zamiast prostych linii w każdym z tych segmentów funkcja jest przybliżana za pomocą

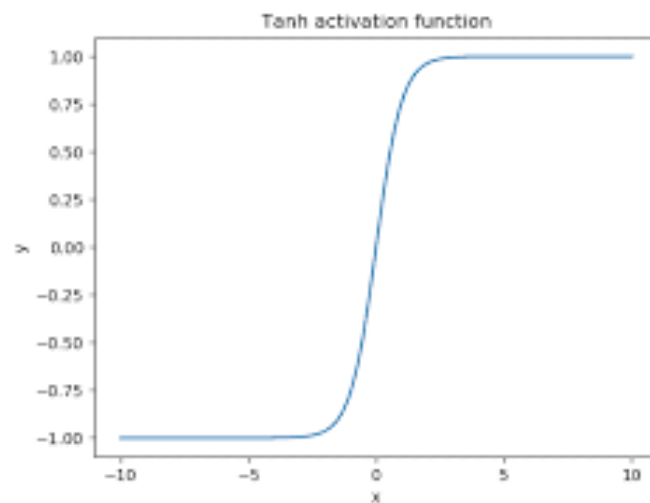
wielomianu stopnia n . Równanie dla każdego segmentu ma postać:

$$p_k(x_i) = a_n x_i^n + a_{n-1} x_i^{n-1} + \dots + a_1 x_i + a_0$$

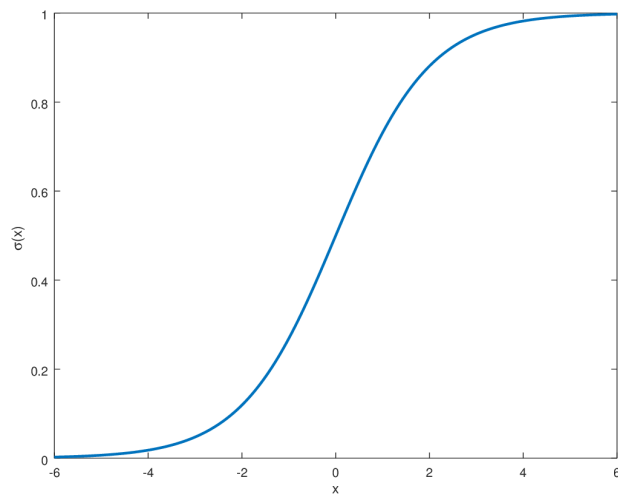
gdzie k to indeks segmentu, a_n to współczynniki wielomianu. Współczynniki dla każdego segmentu są obliczane przy użyciu macierzy Vandermonde'a [2].



Rysunek 1.1: Wykres funkcji ReLU



Rysunek 1.2: Wykres funkcji tanh



Rysunek 1.3: Wykres funkcji sigmoid

2 Skrótowa lista potencjalnych metod

2.1 ReLU

- Prosta implementacja w Verilog'u opisana poniżej

2.2 Sigmoid i tanh

- Implementacja za pomocą LUT
- Implementacja za pomocą LUT z interpolacją
- Implementacja za pomocą CORDIC
- Implementacja za pomocą szeregu Taylora
- Implementacja za pomocą aproksymacji wielomianami Czebyszewa
- Implementacja za pomocą aproksymacji odcinkowo-liniowej (PWL - Piecewise Linear Approximation)
- Implementacja za pomocą aproksymacji odcinkowo-wielomianowej (PWP - Piecewise Polynomial Approximation)
- Implementacja za pomocą aproksymacji odcinkowo-liniowej z ulepszeniami (PLAN - Piecewise Linear Approximation with Optimized Coefficients)

3 Szczegółowy opis metod

Tutaj znajduje się dokładny opis każdej z metod, wraz z przykładami i analizą teoretyczną.

3.1 LUT

Metoda LUT (Look-Up Table) polega na wcześniejszym wyznaczeniu wartości funkcji w środowisku wysokopoziomowym (np. Python), a następnie zapisaniu ich w pamięci ROM implementowanej w FPGA jako bloki BRAM. W trakcie pracy układu sygnał wejściowy nie jest przetwarzany arytmetycznie, lecz bezpośrednio interpretowany jako adres pamięci, z którego odczytywana jest gotowa wartość funkcji.

Adresowanie pamięci odbywa się poprzez przeskalowanie wejścia stałoprzecinkowego (w tym przypadku Q4.12) do zakresu indeksów tablicy. Wejście jest najpierw przesuwane o stałą OFFSET, a następnie odpowiednio redukowane bitowo tak, aby pokrywało zakres od 0 do $2^{AW} - 1$. Dzięki temu cały zakres funkcji (np. od -4.0 do +4.0) jest liniowo mapowany na dostępne próbki LUT. Dodatkowo stosuje się nasycenie na krańcach zakresu, aby uniknąć wyjścia poza zdefiniowaną domenę pamięci.

3.2 LUT + interpolacja liniowa / wielomianowa

LUT z interpolacją liniową lub wielomianową stanowi kompromis pomiędzy podejściem czysto pamięciowym a obliczeniowym. Zamiast przechowywać gęstą tablicę wartości funkcji, stosuje się znacznie rzadszy LUT, w którym wejście jest kwantyzowane do ograniczonej liczby punktów referencyjnych. W prezentowanej implementacji rozmiar tablicy został zredukowany do 64 próbek poprzez zmniejszenie szerokości adresu (AW), co istotnie ogranicza zużycie pamięci kosztem konieczności rekonstrukcji wartości pośrednich.

Wejście jest najpierw przeskalowane i przesunięte do dodatniego zakresu za pomocą stałej OFFSET, a następnie dzielone na część adresową oraz część ułamkową. Starsze bity wejścia wyznaczają indeks bazowy w LUT, natomiast młodsze bity reprezentują pozycję wewnątrz przedziału pomiędzy dwoma sąsiednimi punktami. Dzięki temu możliwe jest odtworzenie wartości funkcji w sposób przybliżony, ale znacznie dokładniejszy niż w przypadku pojedynczego odczytu z tablicy.

Aby umożliwić jednoczesny dostęp do dwóch sąsiednich próbek, LUT został rozdzielony na dwa niezależne banki pamięci (parzysty i nieparzysty). Takie rozdzielenie pozwala w jednym cyklu zegara pobrać zarówno wartość bazową, jak i kolejną próbkę, co eliminuje konieczność dodatkowych opóźnień wynikających z dostępu sekwencyjnego. Dodatkowo najmłodszy bit adresu jest wykorzystywany do wyboru poprawnej kolejności próbek w procesie interpolacji.

Drugi etap potoku obejmuje odczyt danych z pamięci oraz zatrzaśnięcie zarówno wartości próbek, jak i części ułamkowej wejścia. Fragment ułamkowy określa, jak daleko pomiędzy dwoma sąsiednimi punktami znajduje się aktualne wejście i jest wykorzystywany jako współczynnik wagowy w interpolacji. Dzięki temu układ zachowuje ciągłość aproksymowanej funkcji mimo rzadkiego próbkowania LUT.

W kolejnym etapie wykonywana jest interpolacja liniowa, w której obliczana jest różnica pomiędzy dwiema sąsiednimi wartościami funkcji, a następnie mnożona przez część ułamkową wejścia. Wynik tego mnożenia jest odpowiednio przesuwany w prawo, aby uwzględnić skalowanie fixed-point. Ostatecznie do wartości bazowej dodawana jest składowa interpolacyjna, co daje przybliżoną wartość funkcji w punkcie wejściowym.

3.3 Algorytm CORDIC (Coordinate Rotation Digital Computer)

Implementacja CORDIC wykorzystuje iteracyjny algorytm oparty na mikrorotacjach hiperbolicznych. W przeciwieństwie do metod LUT oraz metod aproksymacyjnych, wartości funkcji nie są wcześniej zapisane w pamięci ani nie są odtwarzane na podstawie odcinków liniowych. Wynik jest obliczany arytmetycznie z użyciem dodawania, odejmowania oraz przesunięć bitowych. Dzięki temu implementacja nie wymaga mnożników ani bloków pamięci przechowujących próbki funkcji.

W prezentowanym wariantcie funkcja $\tanh$ jest wyznaczana na podstawie zależności pomiędzy funkcjami hiperbolicznymi. Najpierw za pomocą hiperbolicznego CORDIC

obliczane są wartości proporcjonalne do $\sinh(x)$ oraz $\cosh(x)$, a następnie ich iloraz jest wyznaczany przy użyciu drugiego etapu CORDIC pracującego w trybie dzielenia. Współczynnik skalujący charakterystyczny dla CORDIC nie musi być osobno kompensowany, ponieważ pojawia się zarówno w liczniku, jak i w mianowniku, więc skraca się podczas obliczania ilorazu.

Sygnał wejściowy jest podawany w formacie stałoprzecinkowym Q4.12, a następnie konwertowany do wewnętrznego formatu Q5.16. Zwiększenie liczby bitów ułamkowych pozwala ograniczyć błąd zaokrągleń powstający podczas kolejnych przesunięć i dodawań. Wewnętrzna szerokość słowa została dobrana tak, aby zachować zapas na wartości pośrednie generowane podczas rotacji hiperbolicznych.

Pierwsza część układu realizuje uproszczony hiperboliczny CORDIC. Wektor początkowy ma postać $(1, 0)$, natomiast argument funkcji jest zapisany w zmiennej odpowiadającej pozostałemu kątowi rotacji. W kolejnych krokach znak tej zmiennej decyduje o kierunku mikrorotacji. Jeżeli pozostały kąt jest dodatni, wykonywane jest dodawanie przesuniętych wartości współrzędnych, natomiast dla kąta ujemnego wykonywane jest odejmowanie. Każda mikrorotacja wykorzystuje stałą $\operatorname{atanh}(2^{-i})$, która określa rozmiar wykonywanego kroku.

W implementacji zastosowano niewielką, ręcznie rozwiniętą liczbę mikrorotacji. Kilka pierwszych rotacji wykorzystuje przesunięcie o jeden bit, co zwiększa zakres działania algorytmu dla większych wartości wejściowych. Następnie dodany jest krok z przesunięciem o dwa bity. Takie rozwiązanie nie jest maksymalnie dokładną wersją CORDIC, ale znacząco upraszcza układ i pozwala ograniczyć liczbę zasobów wykorzystywanych podczas syntezy.

Druga część układu realizuje dzielenie CORDIC w trybie liniowym. Do dzielnika trafiają wartości otrzymane z etapu hiperbolicznego, czyli przybliżone $\sinh(x)$ oraz $\cosh(x)$. Dzielnik iteracyjnie zmniejsza wartość reszty, dodając lub odejmując przesuniętą wartość mianownika. Równocześnie aktualizowana jest zmienna wyniku, do której dodawane lub odejmowane są kolejne potęgi dwójki. W ten sposób uzyskiwane jest przybliżenie ilorazu $\sinh(x) / \cosh(x)$, czyli wartości funkcji $\tanh(x)$.

Całość została zapisana bez pętli generujących, bez tablic rejestrowych oraz bez dodatkowych podmodułów. Kolejne kroki CORDIC są rozwinięte bezpośrednio jako połączenia kombinacyjne. Takie podejście zostało przyjęte głównie ze względu na wymagania syntezy, ponieważ wcześniejsze wersje potokowe z wieloma instancjami były zbyt kosztowne dla narzędzia syntezy. Obecna implementacja jest mniej dokładna, ale znacznie prostsza strukturalnie i łatwiejsza do przetworzenia przez syntezytor.

Wynik końcowy jest zaokrąglany i konwertowany z formatu Q5.16 z powrotem do formatu Q4.12. Układ posiada jeden rejestr wyjściowy, dlatego latencja widziana przez testbench wynosi jeden cykl zegara. Oznacza to, że cała część obliczeniowa pracuje kombinacyjnie, a zarejestrowany zostaje dopiero wynik końcowy. Jest to rozwiązanie kompromisowe: ogranicza liczbę rejestrów i upraszcza syntezę, ale może prowadzić do dłuższej ścieżki krytycznej niż w pełni potokowa implementacja CORDIC.

Zaletą tej metody jest zachowanie charakteru czystego CORDIC, ponieważ wynik powstaje wyłącznie na podstawie mikrorotacji, przesunięć bitowych oraz dodawania i odejmowania. Implementacja nie korzysta z tablicy wartości funkcji tanh, interpolacji, aproksymacji odcinkowo-liniowej ani wielomianów. Główną wadą jest ograniczona dokładność wynikająca z małej liczby iteracji oraz uproszczonej struktury, jednak w zamian układ jest znacznie lżejszy i bardziej przyjazny dla procesu syntezy.

3.4 Szereg Taylora

Implementacja za pomocą szeregu Taylora polega na przybliżeniu funkcji tanh wielomianem o skończonej liczbie składników. W przeciwieństwie do metod LUT, układ nie przechowuje wcześniej obliczonych próbek funkcji w pamięci. Wartość wyjściowa jest obliczana arytmetycznie na podstawie aktualnej wartości wejściowej, z wykorzystaniem mnożeń, dodawań, odejmowań oraz stałych współczynników wynikających bezpośrednio z rozwinięcia funkcji w szereg.

W prezentowanej implementacji zastosowano uproszczone rozwinięcie funkcji tanh wokół zera. Wykorzystano pierwsze niezerowe składniki szeregu, dzięki czemu funkcja jest aproksymowana wielomianem nieparzystym. Taka postać jest naturalna dla funkcji tanh, ponieważ funkcja ta jest symetryczna względem początku układu współrzędnych. Oznacza to, że dla argumentów dodatnich i ujemnych zachowywany jest ten sam kształt funkcji z przeciwnym znakiem wyniku.

Sygnal wejściowy jest reprezentowany w formacie stałoprzecinkowym Q4.12. Obliczenia prowadzone są z użyciem rozszerzonej szerokości słowa, aby ograniczyć utratę dokładności podczas mnożeń i przesunięć bitowych. Najpierw wyznaczane są kolejne potęgi argumentu, takie jak składnik kwadratowy oraz czwarta potęga. Następnie są one wykorzystywane do zbudowania wyrażenia wielomianowego odpowiadającego przybliżeniu szeregu Taylora.

Aproksymacja została zapisana w postaci przekształconej, w której wspólny czynnik wejściowy jest wyłączony przed nawias. Dzięki temu zamiast bezpośrednio obliczać

osobno składniki x , x^3 oraz x^5 , układ wyznacza najpierw część zależną od x^2 i x^4 , a następnie mnoży ją przez wejście. Pozwala to zachować strukturę zgodną z szeregiem Taylora, a jednocześnie uprościć zapis operacji arytmetycznych w Verilogu.

Po obliczeniu wartości wielomianu wynik jest ograniczany do zakresu funkcji $\tanh$. Ponieważ szereg Taylora dobrze przybliża funkcję głównie w otoczeniu zera, dla większych wartości argumentu wielomian może dawać wyniki wykraczające poza przedział od -1 do 1. Z tego powodu na końcu zastosowano nasycenie, które zabezpiecza wynik przed przekroczeniem dopuszczalnego zakresu wyjściowego.

Cały układ ma jeden rejestr wyjściowy, dlatego latencja wynosi jeden cykl zegara. Część obliczeniowa jest realizowana kombinacyjnie, a zarejestrowany zostaje dopiero wynik końcowy. Taka struktura jest prosta i czytelna, ale zawiera operacje mnożenia oraz dzielenia przez stałe, co może prowadzić do większego zużycia zasobów niż w metodach opartych wyłącznie na przesunięciach bitowych.

3.5 Aproksymacja wielomianami Czebyszewa

Implementacja za pomocą wielomianów Czebyszewa polega na przybliżeniu funkcji $\tanh$ jako kombinacji liniowej specjalnej bazy wielomianowej. W przeciwieństwie do szeregu Taylora funkcja nie jest rozwijana bezpośrednio w klasycznych potęgach argumentu, takich jak x , x^3 czy x^5 . Zamiast tego wykorzystuje się wielomiany Czebyszewa, które pozwalają lepiej rozłożyć błąd aproksymacji w całym analizowanym przedziale.

W prezentowanej implementacji zakres wejściowy od -4 do 4 jest najpierw przeskalowany do przedziału od -1 do 1 . W tym celu wprowadzana jest zmienna pomocnicza $u = x/4$. Takie przeskalowanie jest istotne, ponieważ wielomiany Czebyszewa są naturalnie definiowane i stosowane właśnie na przedziale $[-1, 1]$. Dzięki temu wejście w formacie Q4.12 może zostać użyte do obliczania wartości kolejnych składników bazy Czebyszewa.

Aproksymacja funkcji została zapisana w postaci kombinacji trzech nieparzystych wielomianów:

$$\tanh(x) \approx c_1 T_1(u) + c_3 T_3(u) + c_5 T_5(u).$$

Wykorzystanie tylko nieparzystych składników wynika z własności funkcji $\tanh$, która jest funkcją nieparzystą. Oznacza to, że jej wykres jest symetryczny względem początku układu współrzędnych, dlatego składniki parzyste nie są potrzebne w uproszczonej aproksymacji.

W układzie obliczane są wartości bazowe $T_1(u)$, $T_3(u)$ oraz $T_5(u)$. Wielomian $T_1(u)$ odpowiada bezpośrednio zmiennej u , natomiast wyższe składniki są wyznaczane na podstawie potęg u^3 oraz u^5 i odpowiednich zależności wielomianów Czebyszewa. Następnie każdy z tych składników jest mnożony przez stały współczynnik aproksymacji. Współczynniki c_1 , c_3 oraz c_5 zostały zapisane w formacie stałoprzecinkowym Q2.14, co umożliwia ich bezpośrednie użycie w arytmetyce fixed-point.

Po przemnożeniu składników przez współczynniki wyniki cząstkowe są ponownie skalowane przez przesunięcia bitowe, aby zachować poprawny format stałoprzecinkowy. Następnie są sumowane, tworząc końcową wartość wielomianu. Otrzymany wynik jest ograniczany do zakresu funkcji tanh, czyli od -1 do 1 . W formacie Q4.12 odpowiada to wartościom od -4096 do 4096 .

Zastosowana wersja ma prostą strukturę kombinacyjną z jednym rejestrem wyjściowym. Oznacza to, że wynik jest zatrzymywany po jednym cyklu zegara, natomiast większość obliczeń wykonywana jest kombinacyjnie. Implementacja wymaga mnożników do wyznaczania potęg oraz mnożenia przez współczynniki, ale nie korzysta z LUT, interpolacji, CORDIC ani aproksymacji odcinkowo-liniowej. Dzięki temu stanowi osobną metodę aproksymacji funkcji tanh.

3.6 Aproksymacja odcinkowo-liniowa (PWL - Piecewise Linear Approximation)

Implementacja PWL polega na podziale krzywej funkcji tanh na kilka prostoliniowych fragmentów. Zamiast obliczać funkcję dokładnie, dla każdego przedziału wejściowego stosowana jest osobna funkcja liniowa postaci:

$$y = a_i x + b_i.$$

Współczynnik a_i określa nachylenie prostej w danym przedziale, natomiast b_i odpowiada za jej przesunięcie. W prezentowanej implementacji współczynniki zostały dobrane na podstawie wartości funkcji tanh w punktach granicznych kolejnych przedziałów, dlatego jest to klasyczna aproksymacja odcinkowo-liniowa, a nie wersja z optymalizowanymi współczynnikami.

Ze względu na nieparzystość funkcji tanh obliczenia wykonywane są tylko dla wartości bezwzględnej wejścia. Na początku układ sprawdza znak argumentu i wyznacza $|x|$. Następnie na podstawie wartości bezwzględnej wybierany jest odpowiedni przedział

aproksymacji. Dzięki temu wystarczy przechowywać współczynniki tylko dla dodatniej części funkcji, a znak wyniku jest odtwarzany dopiero na końcu.

Zakres dodatni został podzielony na kilka przedziałów, między innymi okolice od 0 do 0.5, od 0.5 do 1.0, od 1.0 do 1.5, od 1.5 do 2.0 oraz od 2.0 do 4.0. Dla każdego z tych fragmentów zapisano osobną parę współczynników a_i i b_i . Współczynniki nachylenia są reprezentowane w formacie stałoprzecinkowym Q2.14, natomiast wynik funkcji jest przechowywany w formacie Q4.12.

Po wyborze przedziału wykonywane jest mnożenie wartości wejściowej przez współczynnik nachylenia, a następnie dodawany jest odpowiedni wyraz wolny. Wynik mnożenia jest przesuwany bitowo, aby przywrócić właściwe skalowanie fixed-point. Dla wartości wejściowych znajdujących się poza analizowanym zakresem stosowane jest nasycenie, ponieważ funkcja $\tanh$ asymptotycznie zbliża się do wartości 1 oraz -1 .

Układ posiada jeden rejestr wyjściowy, dlatego latencja wynosi jeden cykl zegara. Część kombinacyjna obejmuje wybór przedziału, wybór współczynników oraz obliczenie równania prostej. W porównaniu z LUT metoda PWL wymaga mniej zapamiętanych wartości funkcji, ale korzysta z arytmetyki mnożenia i dodawania. W porównaniu z CORDIC jest prostsza obliczeniowo, ponieważ nie wymaga wielu iteracyjnych mikro-kroków.

3.7 Aproksymacja odcinkowo-wielomianowa (PWP - Piecewise Polynomial Approximation)

Aproksymacja odcinkowo-wielomianowa stanowi rozszerzenie metody PWL. Podobnie jak w aproksymacji odcinkowo-liniowej, zakres wejściowy funkcji $\tanh$ jest dzielony na kilka przedziałów. Różnica polega na tym, że w każdym przedziale funkcja nie jest opisywana prostą, lecz wielomianem wyższego stopnia. W prezentowanej implementacji zastosowano wielomiany drugiego stopnia postaci:

$$y = a_i x^2 + b_i x + c_i.$$

Współczynniki a_i , b_i oraz c_i są stałe dla danego przedziału. Układ najpierw sprawdza, w którym zakresie znajduje się wartość wejściowa, a następnie wybiera odpowiedni zestaw współczynników. W odróżnieniu od PWL pojedynczy odcinek może lepiej odwzorować krzywiznę funkcji $\tanh$, ponieważ zawiera składnik kwadratowy.

Ze względu na nieparzystość funkcji $\tanh$ obliczenia są wykonywane dla wartości bezwzględnej argumentu. Na początku wyznaczane jest $|x|$, a znak wejścia jest zapamiętywany i używany dopiero na końcu do odtworzenia znaku wyniku. Dzięki temu współczynniki wielomianów muszą opisywać tylko dodatnią część funkcji, co upraszcza implementację.

W prezentowanym wariantcie dodatni zakres wejściowy został podzielony na kilka przedziałów. Dla każdego z nich dobrano osobny wielomian kwadratowy. Wartość x^2 jest obliczana przez mnożenie wejścia przez samo siebie, a następnie odpowiednio przesuwana bitowo, aby zachować format stałoprzecinkowy. Następnie układ oblicza składniki $a_i x^2$ oraz $b_i x$, po czym dodaje wyraz wolny c_i .

Współczynniki nachylenia i składnika kwadratowego są zapisane w formacie stałoprzecinkowym Q2.14, natomiast wejście i wynik funkcji są reprezentowane w formacie Q4.12. Po wykonaniu mnożeń wyniki cząstkowe są skalowane przez przesunięcia bitowe, aby wrócić do właściwego formatu fixed-point. Na końcu stosowane jest nasycenie do zakresu funkcji $\tanh$, czyli od -1 do 1 .

Układ posiada jeden rejestr wyjściowy, dlatego latencja wynosi jeden cykl zegara. W porównaniu z PWL metoda PWP wymaga większej liczby operacji arytmetycznych, ponieważ konieczne jest obliczenie składnika kwadratowego oraz wykonanie dodatkowego mnożenia. W zamian pozwala lepiej odwzorować nieliniowy kształt funkcji przy mniejszej liczbie przedziałów.

3.8 Metoda PLAN (Piecewise Linear Approximation of Nonlinear Functions)

Metoda PLAN jest sprzętowo zoptymalizowaną odmianą aproksymacji odcinkowo-liniowej. Podobnie jak w klasycznym PWL, zakres wejściowy funkcji $\tanh$ jest dzielony na kilka przedziałów, a każdy z nich jest opisywany równaniem prostej. Różnica polega na sposobie doboru współczynników nachylenia. W metodzie PLAN nachylenia są dobierane tak, aby były potęgami dwójki, na przykład 1 , $1/2$, $1/4$, $1/8$ lub $1/32$.

Dzięki takiemu doborowi współczynników operacja mnożenia przez nachylenie nie wymaga sprzętowego mnożnika. W Verilogu mnożenie przez potęgę dwójki można zastąpić przesunięciem bitowym w prawo. Oznacza to, że wyrażenie liniowe w danym przedziale nie jest realizowane jako ogólne $a_i x + b_i$, lecz jako dodanie przesuniętej wartości różnicy względem początku przedziału.

W prezentowanej implementacji najpierw wyznaczana jest wartość bezwzględna wejścia. Jest to możliwe dzięki temu, że funkcja $\tanh$ jest funkcją nieparzystą. Układ aproksymuje tylko dodatnią część charakterystyki, a znak wyniku jest odtwarzany na końcu. Takie rozwiązanie zmniejsza liczbę wymaganych przedziałów i upraszcza logikę wyboru.

Dodatni zakres wejścia został podzielony na kilka fragmentów. Dla małych wartości argumentu zastosowano przybliżenie o nachyleniu równym jeden, ponieważ w pobliżu zera funkcja $\tanh$ jest zbliżona do funkcji liniowej. W kolejnych przedziałach nachylenie jest stopniowo zmniejszane przez użycie przesunięć bitowych, co odzwierciedla spłaszczanie się funkcji $\tanh$ dla większych wartości argumentu.

Dla każdego przedziału układ oblicza różnicę między wartością wejściową a początkiem danego przedziału. Następnie różnica ta jest przesuwana w prawo o ustaloną liczbę bitów i dodawana do wartości bazowej odpowiadającej początkowi przedziału. W ten sposób powstaje prosta aproksymująca dany fragment funkcji. Dla dużych wartości argumentu stosowane jest nasycenie, ponieważ funkcja $\tanh$ asymptotycznie zbliża się do wartości 1.

Cała implementacja składa się z dekodera przedziałów, kilku odejmowań, przesunięć bitowych, dodawań oraz jednego rejestru wyjściowego. Nie wykorzystuje bloków DSP ani pamięci z próbkami funkcji. Latencja układu wynosi jeden cykl zegara, ponieważ część obliczeniowa jest kombinacyjna, a wynik jest rejestrowany na wyjściu.

4 Implementacja

4.1 ReLU

```
1 module relu (  
2     input wire signed [31:0] din_relu,  
3     output wire signed [31:0] dout_relu  
4 );  
5     assign dout_relu = (din_relu < 0) ? 32'b0 : din_relu;  
6 endmodule
```

Listing 4.1: Implementacja funkcji ReLU w Verilog'u [1]

Pozostałe implementacje zajmują już więcej miejsca, dlatego w celu zachowania czytelności dokumentu, poniżej znajdują się tylko odnośniki do kodu źródłowego, który jest dostępny w repozytorium GitHub.

4.2 tanh

- **Implementacja za pomocą LUT**
w pliku `implementations/tanh/LUT_tanh.sv`
- **Implementacja za pomocą LUT z interpolacją**
w pliku `implementations/tanh/interpolated_LUT_tanh.sv`
- **Implementacja za pomocą CORDIC**
w pliku `implementations/tanh/CORDIC.sv`
- **Implementacja za pomocą Taylor'a**
w pliku `implementations/tanh/Taylor.sv`
- **Implementacja za pomocą Czebyszewa**
w pliku `implementations/tanh/Czebyszew.sv`

- **Implementacja za pomocą PWL**
w pliku `implementations/tanh/PWL.sv`
- **Implementacja za pomocą PWP**
w pliku `implementations/tanh/PWP.sv`
- **Implementacja za pomocą PLAN**
w pliku `implementations/tanh/PLAN.sv`

5 Github

Wszystkie implementacje, testbenche oraz aktualna wersja dokumentacji będą dostępne w publicznym repozytorium na GitHubie pod adresem <https://github.com/KowalczykPatryk/ActivationFunctions>.

6 Testbench

Tutaj znajdują się odnośniki do kodu testbencha, które będą używane do weryfikacji poprawności działania wszystkich implementacji. Testbench będzie automatycznie generował różne wartości wejściowe i porównywał wyniki z idealnymi wartościami obliczonymi w Pythonie. Obliczał będzie również metryki błędu, takie jak błąd maksymalny, MAE i RMSE. Dodatkowo liczył będzie opóźnienie (latency) zliczając rising edge zegara od momentu pojawienia się sygnału wejściowego do momentu pojawienia się prawidłowego wyniku na wyjściu.

W teście do modułu podawany jest zbiór 2000 wartości wejściowych równomiernie rozmieszczonych w przedziale $x \in [-4, 4]$, zakodowanych w formacie fixed-point U2 Q4.12 na wejściu i wyjściu. Dodatkowo do wyznaczenia opóźnienia (latency) podawana jest próbka kontrolna $x = 1.0$.

6.1 ReLU

- w pliku `testbench/ReLU/testbench.sv`

6.2 tanh

- w pliku `testbench/tanh/testbench.py`

7 Porównywanie metod

Opis w jaki sposób zamierzamy porównać ze sobą wszystkie opisane wcześniej metody implementacji.

7.1 Planowane środowisko symulacyjne i model referencyjny

Wszystkie moduły sprzętowe zostaną napisane w języku Verilog. Do ich uruchomienia, symulacji oraz stworzenia środowisk testowych (ang. *testbench*) planujemy wykorzystać darmową platformę internetową EDA Playground (<https://www.edaplayground.com>).

Testbench standardowo jest pisany w Verilogu, ale w EDA Playground można też użyć Python'a. Oznacza to użycie frameworka Cocotb. Pozwala on na pisanie testów układów sprzętowych w Pythonie, podczas gdy sam układ napisany jest w Verilogu.

Działanie testbencha będzie polegać na automatycznym podawaniu na wejście układu RTL całego zakresu zmiennych i zapisywaniu tego, co układ zwróci na wyjściu. Żeby sprawdzić jakość działania, wyniki z Veriloga będą porównywane z idealną teorią matematyczną. Jako idealny wzorzec (referencję) zamierzamy wykorzystać obliczenia wykonane w Pythonie na liczbach zmiennoprzecinkowych.

7.2 Kryteria oceny

Każdą z metod będę oceniać pod kątem trzech głównych parametrów: dokładności, zużycia zasobów oraz szybkości działania.

7.2.1 Dokładność (Analiza błędu)

Do zmierzenia błędu planujemy wykorzystać trzy metryki:

- **Błąd maksymalny** – pokaże największą różnicę między teorią a Verilogiem, jaka przytrafi się w całym teście. Wskaże to na „najgorszy możliwy przypadek”.
- **Średni błąd bezwzględny (MAE)** – pokaże, o ile średnio myli się projektowany układ dla wszystkich podanych próbek.
- **Pierwiastek błędu średniokwadratowego (RMSE)** – podobnie jak MAE pokaże błąd, ale znacznie mocniej ukarze duże, pojedyncze odchylenia od wykresu.

7.2.2 Zużycie zasobów FPGA

Z raportów po syntezie układu będą odczytywane dane o tym, ile miejsca w strukturze FPGA zajmie dana metoda. Sprawdzane będą trzy kluczowe elementy:

- **LUT** – (Lookup Table) to podstawowy blok logiczny FPGA realizujący dowolną funkcję logiczną. Ich liczba powie o tym, jak skomplikowana logicznie jest dana metoda.
- **FF (Przerzutniki)** – elementy pamięciowe czyli przerzutniki przechowujące stan między taktami zegara. W Siemens nazywane są DFF czyli D-type FF (Flip-Flops).
- **DSP** – wbudowane w FPGA szybkie sprzętowe mnożniki. Ważnym celem projektowym będzie ich oszczędzanie, ponieważ ich liczba w układach FPGA jest mocno ograniczona. W Xilinx Artix-7 7A100TCSG324 jest ich 240, a dla porównania LUT’ów jest 63400, przerzutników jest 126800.

7.2.3 Szybkość działania

Wydajność czasowa układów zostanie sprawdzona na podstawie dwóch parametrów:

- **Maksymalna częstotliwość (F_{max})** – będzie informować o tym, z jak szybkim zegarem (w MHz) może maksymalnie pracować dany moduł, żeby nie doszło do błędów.
- **Opóźnienie na ścieżce krytycznej (Critical Path Delay)** – będzie informować o tym, ile czasu (w nanosekundach) zajmuje przetworzenie jednej próbki od

momentu pojawienia się na wejściu do momentu pojawienia się prawidłowego wyniku na wyjściu. To całkowity czas najwolniejszej ścieżki w implementacji po routingu. W praktyce jest to najważniejsza liczba timingowa, bo mówi, jak długi musi być okres zegara bez naruszania timingów.

- **Opóźnienie (ang. *Latency*)** – czyli liczba cykli zegara, jaką układ będzie potrzebował od momentu wrzucenia liczby na wejście do momentu, gdy prawidłowy wynik pojawi się na wyjściu.

7.3 Narzędzia w EDA Playground

7.3.1 Symulatory (ang. *simulators*)

Służą do uruchamiania kodu napisanego w języku opisu sprzętu Verilog oraz wykonywania testbencha. Wykonywanie symulacji czasowej oraz analizę sygnałów i przebiegów.

Komercyjne symulatory dostępne w EDA Playground:

- Aldec Riviera Pro
- Cadence Xcelium
- Siemens Questa (ten wybrany)
- Synopsys VCS

Darmowe symulatory dostępne w EDA Playground:

- GHDL
- Verilator
- Icarus Verilog

7.3.2 Syntezatory (ang. *synthesizers*)

Przekształcają opis układu zapisany w języku HDL na rzeczywistą strukturę sprzętową, składającą się z bramek logicznych, przerzutników i innych elementów. Proces ten

pozwała określić, jak projekt zostanie zaimplementowany w układzie FPGA.

Komercyjne synteзаторы dostępne w EDA Playground:

- Siemens Precision (ten wybrany)

Darmowe synteзаторы dostępne w EDA Playground:

- Yosys

Synteza została przeprowadzona przy użyciu Siemens Precision 2024.2 dla układu Xilinx Artix-7 7A100TCSG324.

7.3.3 VTR 7.0

Oprócz tego co synteзатор czyli obliczania rzeczywistej struktury sprzętowej, VTR (ang. *Verilog-to-Routing*) to narzędzie robi również placing (umieszczanie elementów na układzie) oraz routing (łączenie ich ze sobą). W EDA Playground to narzędzie nie jest już wspierane i jest trochę przestarzałe, więc aby dało się uruchomić trzeba użyć starszej składni veriloga.

7.4 Co sprawdzane w czym

7.4.1 W Pythonie z Cocotb

- Porównanie z idealną teorią matematyczną. Gdy symulator przetwarza kolejne próbki, Cocotb na bieżąco porównuje wyniki z Veriloga z wynikami z Pythona.
- Analiza błędu (Max error, MAE, RMSE).
- Opóźnienie (Latency). Cocotb potrafi dokładnie liczyć takty zegara. Mierzy się to licząc rising edge w trakcie działania.

7.4.2 W Siemens Precision

- Zużycie zasobów FPGA (LUT, FF, DSP)
- Maksymalna częstotliwość (F_{max})

- Opóźnienie na ścieżce krytycznej (Critical Path Delay)

Tutaj żeby dostać maksymalną częstotliwość oraz opóźnienie na ścieżce krytycznej trzeba ustawić okres zegara w constraints (np. 1ns) i wtedy syntezytor będzie próbował się do tego dostosować. Celem jest aby mu się to nie udało bo wtedy dostajemy ujemny slack, który jest różnicą między tym, co zaprojektowaliśmy (1ns) a tym, co jest w stanie osiągnąć nasz układ (np. 1.2ns). Teraz mając ujemny slack, możemy obliczyć maksymalną częstotliwość jako $F_{max} = \frac{1}{T_{clock} - slack}$, gdzie T_{clock} to okres zegara (1ns). Opóźnienie na ścieżce krytycznej to po prostu $T_{clock} - slack$.

7.4.3 W VTR 7.0 (tylko dla prostego ReLU)

Problem z aktualnością pojawił się już przy implementacji LUT.

- Maksymalna częstotliwość (F_{max})
- Opóźnienie na ścieżce krytycznej (Critical Path Delay)

8 Wyniki

8.1 ReLU

Układ jest bardzo prosty ale dla testu został napisany w Verilogu testbench który podaje na wejście 32-bitowe liczby całkowite ze znakiem i sprawdza, czy na wyjściu pojawia się poprawna wartość (0 dla liczb ujemnych i ta sama liczba dla liczb nieujemnych). Wyjście jest poprawne dla wszystkich testowanych wartości, a log jest zapisany w pliku testbench/ReLU/testbench_results.txt.

Tabela 8.1: Podsumowanie wyników liczbowych

LUT	FF	DSP	F_{max} [MHz]	CPD [ns]
16	0	0	944.411	1.05886

gdzie:

LUT – liczba użytych tablic LUT,
FF – liczba użytych przerzutników,
DSP – liczba użytych bloków DSP,
 f_{max} – maksymalna częstotliwość,
CPD – critical path delay.

8.2 tanh

Tabela 8.2: Podsumowanie wyników liczbowych (tanh)

Metoda	MaxE	MAE	RMSE	LUT	FF	DSP	f_{max} [MHz]	Lat.	CPD [ns]
LUT1024	0.004109	0.000510	0.000926	20	1	0	163	1	6.119
LUT1024 inte.	0.004146	0.000947	0.001577	32	17	1	119	3	8.370
LUT64	0.118265	0.014636	0.028037	18	1	0	163	1	6.119
LUT64 inte.	0.004706	0.000967	0.001647	46	43	1	119	3	8.370
CORDIC	0.287157	0.034901	0.065214	373	7	0	36	1	27.329
Taylor	0.153798	0.026829	0.045678	453	14	5	188	1	5.306
Czebyszew	0.097035	0.038416	0.047138	136	14	8	188	1	5.306
PWL	0.025998	0.009809	0.011267	42	16	1	187	1	5.335
PWP	0.010813	0.002501	0.003295	53	13	3	141	1	7.094
PLAN	0.056405	0.010817	0.018035	63	13	0	189	1	5.299

gdzie:

MaxE – błąd maksymalny,

MAE – średni błąd bezwzględny,

RMSE – pierwiastek błędu średniokwadratowego,

LUT – liczba użytych tablic LUT,

FF – liczba użytych przerzutników,

DSP – liczba użytych bloków DSP,

 f_{max} – maksymalna częstotliwość,

Lat. – latency,

CPD – critical path delay.

9 Wnioski

Na początku przetestowano różnice w działaniu między LUT i interpolated LUT. Z powodu wyboru bardzo gęstego LUT (1024 punktów) oba układy osiągnęły bardzo wysoką dokładność, ale interpolated LUT zużył więcej zasobów (LUT, FF, DSP) i miał niższe F_{max} oraz wyższy CPD. Dlatego w dalszej części testów jest użycie rzadszego LUT, aby lepiej zobaczyć różnicę między tymi dwoma metodami.

Po przetestowaniu rzadszego LUT (64 punkty) widać, że dokładność spadła drastycznie, ale interpolated LUT nadal utrzymał bardzo dobrą dokładność, co pokazuje, że interpolacja jest skutecznym sposobem na poprawę jakości przy mniejszej liczbie próbek. Jednakże kosztem tego jest znacznie większe zużycie zasobów i niższa wydajność czasowa.

Implementacja CORDIC okazała się mniej dokładna niż metody oparte na LUT oraz LUT z interpolacją. Wynika to z celowego uproszczenia układu, tak aby możliwe było poprawne przejście syntezy. Ograniczona liczba mikrorotacji zmniejszyła złożoność projektu, ale jednocześnie pogorszyła jakość aproksymacji.

Zaletą tej metody jest brak wykorzystania bloków DSP oraz brak konieczności przechowywania próbek funkcji w pamięci. Jednocześnie układ ma dłuższą ścieżkę kombinacyjną, co negatywnie wpływa na wydajność czasową. CORDIC może być więc korzystny, gdy ważne jest unikanie mnożników, jednak w tej uproszczonej wersji stanowi kompromis między synteżowalnością, dokładnością i szybkością działania.

Implementacja oparta na szeregu Taylora jest prosta koncepcyjnie i nie wymaga pamięci z próbkami funkcji. Dobrze odwzorowuje zachowanie $\tanh$ w pobliżu zera, jednak dla większych wartości argumentu dokładność spada, dlatego konieczne jest zastosowanie nasycenia. Metoda ta stanowi kompromis między prostotą opisu matematycznego a kosztem arytmetyki, ponieważ wymaga operacji mnożenia.

Aproksymacja wielomianami Czebyszewa jest odrębna od szeregu Taylora, ponieważ wykorzystuje inną bazę wielomianową i przeskalowanie wejścia do przedziału $[-1, 1]$. Metoda pozwala uzyskać dobre rozłożenie błędu w całym zakresie, ale wymaga

operacji mnożenia i odpowiedniego skalowania fixed-point. Jest to więc kompromis między dokładnością aproksymacji a kosztem sprzętowym wynikającym z arytmetyki wielomianowej.

Aproksymacja PWP jest dokładniejszym rozwinięciem PWL, ponieważ w każdym przedziale wykorzystuje wielomian zamiast prostej. Pozwala lepiej odwzorować krzywiznę funkcji $\tanh$, ale wymaga większego kosztu sprzętowego ze względu na dodatkowe mnożenia. Jest to kompromis między dokładnością aproksymacji a zużyciem zasobów arytmetycznych.

Metoda PLAN jest bardzo korzystna sprzętowo, ponieważ zastępuje mnożenie przez współczynniki prostych prostymi przesunięciami bitowymi. W porównaniu z klasycznym PWL może być mniej elastyczna pod względem doboru nachyleń, ale znacząco upraszcza implementację i ogranicza zużycie zasobów. Jest to dobry kompromis, gdy ważniejsza jest prostota układu niż maksymalna dokładność aproksymacji.

Bibliografia

- [1] *Activation Function*. URL: <https://thedatabus.in/activation-function/> (term. wiz. 03.06.2026).
- [2] Griselda González-Díaz_Conti i in. „Hardware-Based Activation Function-Core for Neural Network Implementations”. W: (2021). URL: <https://www.mdpi.com/2079-9292/11/1/14> (term. wiz. 03.06.2026).
- [3] Barrie Hayes-Gill. „Piecewise linear approximation applied to nonlinear function of a neural network”. W: (1997). URL: https://www.academia.edu/50788608/Piecewise_linear_approximation_applied_to_nonlinear_function_of_a_neural_network (term. wiz. 03.06.2026).
- [4] Zerun Li i in. „FPGA Implementation for the Sigmoid with Piecewise Linear Fitting Method Based on Curvature Analysis”. W: (2022). URL: <https://www.mdpi.com/2079-9292/11/9/1365> (term. wiz. 03.06.2026).
- [5] Bryon Moyer. *Implementing AI Activation Functions*. URL: <https://semiengineering.com/implementing-ai-activation-functions/> (term. wiz. 03.06.2026).
- [6] *PWL Model*. URL: https://www.bohrium.com/en/sciencepedia/feynman/keyword/pwl_model (term. wiz. 03.06.2026).
- [7] Miroslav Vlcek. „CHEBYSHEV POLYNOMIAL APPROXIMATION FOR ACTIVATION SIGMOID FUNCTION”. W: (2012). URL: <https://www.nnw.cz/doi/2012/NNW.2012.22.023.pdf> (term. wiz. 03.06.2026).